Низкоуровневый эквивалент построен и согласован, что дальше?

Чтобы ответить на вопрос, поставленный в заголовке текста, сделаем два действия:

1) Посмотрим на формулировку персонального задания, которая одинакова для всех вариантов и размещена в начале перечня всех вариантов персональных заданий, содержащегося в файле variantn.rtf внутри архива с исходными данными для КР, переданного в группы в начале семестра. Вот она:

«**Необходимо выполнить доработку элементов макета учебной системы программирования до уровня, позволяющего обрабатывать «новые» для макета конструкции языка высокого уровня, примененные в соответствующем варианте**».

2) Для того, чтобы полнее раскрыть смысл лаконичной формулировки персонального задания, приведенной выше, вспомним про учебное пособие [1] (далее в тексте «**Пособие**»), доступного в библиотеке СПбПУ и также переданного в группы внутри архива с исходными данными для КР, раскроем его, найдем в оглавлении **Пособия** раздел «**Рекомендации по порядку выполнения работы**» и прочитаем следующее:
« . . .
Данная лабораторная работа предполагает использование в качестве начальной основы отлаженной и работающей программы-макета компилятора с языка ПЛ1, с ограниченным набором языковых конструкций, достаточным для написания законченной правильной программы контрольного примера. Программа-макет написана на алгоритмическом языке Си и снабжена подробным контекстным комментарием.

Студенту предлагается расширить первичный набор языковых конструкций, поддерживаемый макетным компилятором с ЯВУ, до уровня, позволяющего оттранслировать пример, содержащийся в его личном задании, и произвести соответствующую доработку макетного компилятора с ЯВУ. При этом необходимо:

а) Ознакомиться с исходными текстами программы-макета и контрольного примера, на который настроена программа-макет,

б) Проверить работоспособность программы-макета путем проведения компиляции контрольного примера,

в) Визуально выделить из исходного текста примера, содержащегося в личном задании, множества языковых конструкций, отсутствующих в контрольном примере,

г) Спроектировать перечень коррекций элементов БД и подпрограмм программы-макета с целью обеспечения компиляции примера из личного задания,

д) Ввести коррекции в исходный текст программы-макета, снабдив измененные места исходного текста построчным комментарием, и записать модифицированный вариант в свой личный директорий на сервере,

е) Выполнить отладку модифицированной программы-макета,

ж) Составить отчет по результатам работы

. . . »

Переходим к выполнению рекомендаций из пп. от **а)** до **ж).**

Действия по пункту *а)*

В архиве с исходными данными для КР находим папку:
StudLabRab_2020_magistry\LinuxLabRab\LinuxLabRab.TGZ\LinuxLabRab.tar\.\ linux_lab_rab

Убеждаемся, во-первых, в том, что в этой папке имеется С-файл **Komppl.c.** Делая беглый входной контроль содержимого файла **Komppl.c**, убеждается в том, что это и есть исходный текст С-программы макета компилятора с языка высокого уровня PL/1, который должен быть доработан исполнителями КР под свой персональный вариант задания.
Далее находим PL/1-файл **examppl.pli** контрольного или, иначе, демо-примера и знакомимся с ним:

```
EXAMP1: PROC OPTIONS ( MAIN );

      DCL A BIN FIXED ( 31 ) INIT (  11B );
      DCL B BIN FIXED ( 31 ) INIT ( 100B );
      DCL C BIN FIXED ( 31 ) INIT ( 101B );
      DCL D BIN FIXED ( 31 );

      D = A + B - C;

END EXAMP1;
```

Рис.1

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту *б)*

Устанавливаем на компьютере ОС Linux для 32-хразрядной IBM PC, например:

- Устанавливаем дистрибутив **ubuntu 14.0**,
- Дополняем пакетом панельного файл-менеджера **Midnight Comander (mc)** с помощью директивы **sudo apt-get install mc**,
- Дополняем пакетом формирования псевдографических окон **ncurses** с помощью директивы **sudo apt-get install libncurses5-dev**,
- Переписываем туда архив с исходными данными для КР, включая папку **linux_lab_rab** .

Для перекодировки исходных файлов из **KOI-8** в **UNICOD (UTF8)** выполняем shell-скрипт **ChangeCodTable**.
Выполняем ту часть shell-скрипта **GenSysProg** (генерация системы программирования), которая относится к построению из исходного С-файла **Komppl.c** исполняемого файла **Komppl.exe**.
Выполняем ту часть shell-скрипта **StartTestTask**, которая относится к запуску компилятора с ЯВУ (т.е. файла **Komppl.exe)** для компиляции демо-примера (т.е. файла **examppl.pli**).

Убеждаемся в том, что эквивалент демо-примера на Ассемблере **examppl.ass** построен компилятором успешно и что он содержит следующий текст на Ассемблере:

Метка	КОП	Операнды	Пояснения
ex	START	0	счетчик отн. адреса в нуль
	BALR	RBASE,0	отн. адр. базы в RBASE
	USING	*,RBASE	Объявл. RBASE регистром базы
	L	RRAB,A	Загрузка A в регистр RRAB
	A	RRAB,B	A+B грузим в RRAB
	S	RRAB,C	A+B-C грузим в RRAB
	ST	RRAB,D	A+B-C из RRAB грузим в D
	BCR	15,RVIX	Переход по адр. в рег. RVIX
A	DC	F’3’	A=3
B	DC	F’4’	B=4
C	DC	F’5’	C=5
D	DS	F	Резерв памяти для D
RBASE	EQU	5	RBASE назначим 5
RRAB	EQU	3	RRAB назначим 3
RVIX	EQU	14	RVIX назначим 14
	END		Конец текста блока

Рис.2

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

<u>Действия по пункту</u> в)

Продолжение настоящих рекомендаций основано на специально построенном для этой цели условном варианте персонального задания, приведенном ниже и имеющем в своей основе демо-пример, приведенный выше (см. рис. 1):

<u>Условный персональный вариант</u>

```
EXAMP1: PROC OPTIONS ( MAIN );
      DCL A BIN FIXED ( 31 ) INIT (  11B );
      DCL B BIN FIXED ( 31 ) INIT ( 100B );
      DCL C BIN FIXED ( 31 ) INIT ( 101B );
      DCL D BIN FIXED ( 31 );
      DCL E BIN FIXED ( 31 );
      D = A + B - C;
      E = D * A / B;
END EXAMP1;
```

Рис.3

При визуальном сравнении исходных текстов условного персонального задания (см. рис. 3) и демо-примера (см. рис. 1) отмечаем то, что отличия в них минимальны и состоят в расширении:

- терминального алфавита грамматики ЯВУ PL/1 путем введения в него новых терминальных символов «*****» и «/»,
- перечня допустимых арифметических операций, применяемых в арифметическом выражении, находящемся в правой части оператора присваивания, операцией **умножения «*»** и операцией **деления «/»**.

Понятно, что отличия, обнаруженные в исходных текстах условного персонального задания и демо-примера, обусловят отличия их эквивалентов на языке Ассемблер. На рис. 4 приведен один из возможных вариантов такого эквивалента для условного персонального задания, приведенного на рис.3.

,

Метка	КОП	Операнды	Пояснения
ex	START	0	счетчик отн. адреса в нуль
	BALR	RBASE,0	отн. адр. базы в RBASE
	USING	*,RBASE	Объявл. RBASE регистром базы
	L	RRAB,A	Загрузка A в регистр RRAB
	A	RRAB,B	A+B грузим в RRAB
	S	RRAB,C	A+B-C грузим в RRAB
	ST	RRAB,D	A+B-C из RRAB грузим в D
	L	RRAB1+1,D	Загрузка D в регистр RRAB1+1
	M	RRAB1,A	D*A грузим в RRAB1 и RRAB1+1
	D	RRAB1,B	D*A/B→RRAB1+1
	ST	RRAB1+1,E	D*A/B из RRAB1+1 грузим в E
	BCR	15,RVIX	Переход по адр. в рег. RVIX
A	DC	F’3’	A=3
B	DC	F’4’	B=4
C	DC	F’5’	C=5
D	DS	F	Резерв памяти для D
E	DS	F	Резерв памяти для E
RBASE	EQU	5	RBASE назначим 5
RRAB	EQU	3	RRAB назначим 3
RVIX	EQU	14	RVIX назначим 14
RRAB1	EQU	6	RRAB1 назначим 6
	END		Конец текста блока

Рис. 4

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

<u>Действия по пункту</u> г)

В основу функционирования макета компилятора с ЯВУ положен (см. **Пособие**) алгоритм, настраиваемый на грамматику ЯВУ, которая выступает в качестве параметра настройки. Грамматика имеет три группы параметров, определяющих такие свойства ЯВУ, как:

- Лексика,
- Синтаксис,
- Семантика.

Детальное пояснение можно найти в **Пособие**, а в данных кратких рекомендациях для пояснений воспользуемся следующим рисунком (см. стр.13 **Пособия)** :

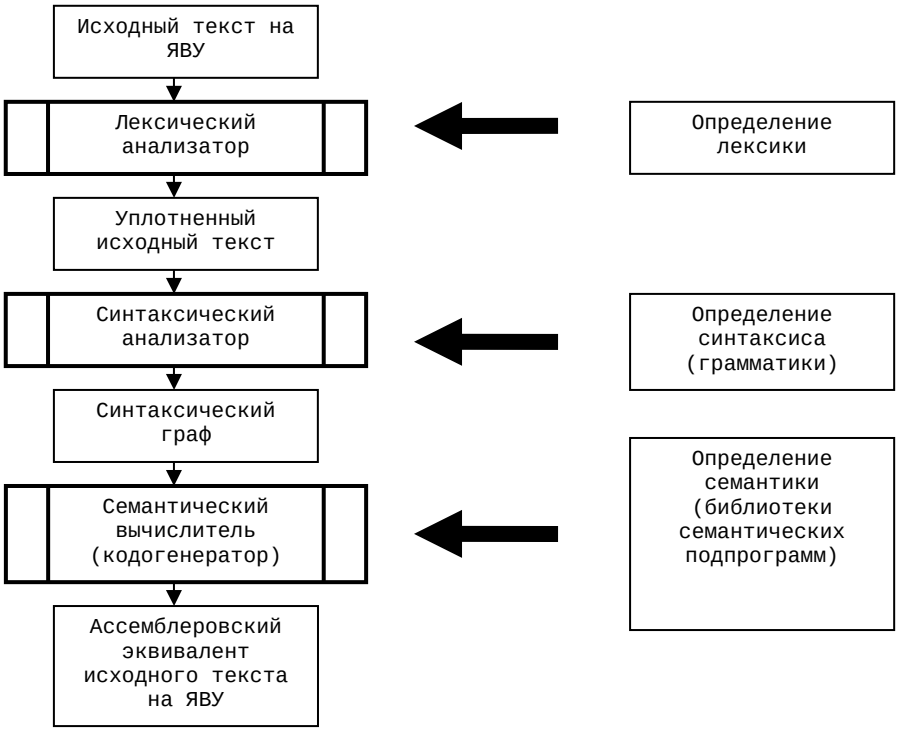

Рис.5. Уточненная функциональная схема компилятора с ЯВУ

Макет компилятора ЯВУ, переданный в группы в архивном файле, уже настроен соответствующими параметрами на грамматику демо-примера в части лексических, синтаксических и семантических аспектов. Все эти параметры размещены, как в элементах БД (лексика синтаксис), так и в множестве семантических функций/подпрограмм (семантика) компилятора с ЯВУ.
Для перенастройки на условный персональный вариант КР необходима коррекция существующих настроек компилятора ЯВУ путем изменения некоторых параметров. Возникает вопрос: «Каких именно?». Проведем экспресс-анализ результатов поиска отличий демо-примера и условного персонального варианта, полученных выше (при выполнении **действий по пункту в**).

Там нами были отмечены отличия условного персонального задания от демо-примера в части расширений:

- терминального алфавита (это лексический аспект ЯВУ),
- перечня допустимых арифметических операций (это синтаксический аспект).

Ответить на вопрос о необходимости коррекции семантических параметров или отсутствии таковой можно будет только после проведения предстоящих коррекций синтаксиса. При этом, критерием необходимости семантических правок будет введение в синтаксис дополнительных нетерминальных символов.

Невозможно, да и методически неправильно в рамках кратких настоящих рекомендаций разбирать детали довольно большой программы макета компилятора ЯВУ с целью указания мест расположения параметров для внесения в них коррекций. Это входит в работу студентов. Поэтому, говоря о коррекции конкретного параметра, будем обсуждать смысловой аспект коррекции и указывать на места описания особенностей корректируемого параметра в **Пособие.** Однако при этом заметим то, что, если единственным местом хранения исходного текста программы макета компилятора ЯВУ является файл **Komppl.c**, то внутри него содержатся и значения всех настроечных параметров.

<u>Лексические правки</u>

Дополнительные терминальные символы «*****» и «/» являются разделителями термов (см. стр. 12 и стр. 13 **Пособия**) и должны быть включены в существующее множество допустимых разделителей:

```
{ ":" , "+" , "-" , ";" , "(" , ")" , "_" }
```

Модифицированное множество допустимых разделителей будет таким:

```
{ ":" , "+" , "-" , ";" , "(" , ")" , "_" , “*” , “/”}
```

<u>Синтаксические правки</u>

Синтаксис демо-примера (см. стр. 7 и стр.8 **Пособия**) имеет следующее определение (см. рис. 6):

```
1.  <PRO> ::= <OPR><TEL><OEN>

2.  <OPR> ::= <IPR>:PROC_OPTIONS(MAIN);

3.  <IPR> ::= <IDE>

4.  <IDE> ::= <BUK> | <IDE><BUK> | <IDE><CIF>

5.  <BUK> ::= A | B | C | D | E | M | P | X

6.  <CIF> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

7.  <TEL> ::= <ODC> | <TEL><ODC> | <TEL><OPA>

8.  <ODC> ::= DCL_<IPE>_BIN_FIXED(<RZR>); |
```

```
                DCL_<IPE>_BIN_FIXED(<RZR>)INIT(<LIT>);

 9.  <IPE> ::= <IDE>

10.  <RZR> ::= <CIF> | <RZR><CIF>

11.  <LIT> ::= <MAN>B

12.  <MAN> ::= 1 | <MAN>0 | <MAN>1

13.  <OPA> ::= <IPE>=<AVI>;

14.  <AVI> ::= <LIT> | <IPE> | <AVI><ZNK><LIT> |

                                        <AVI><ZNK><IPE>

15.  <ZNK> ::= + | -

16.  <OEN> ::= END_<IPR>
```

Здесь использованы следующие метасимволы и символы:

"<" и ">" - левый и правый ограничители нетерминального символа,

"::=" - метасимвол со смыслом "равно по определению",

"|" - метасимвол альтернативного определения "или",

"_" - терминальный символ со смыслом "пробел",

"<PRO>" - нетерминал "программа",

"<OPR>" - нетерминал "оператор пролога программы",

"<IPR>" - нетерминал "имя программы",

"<IDE>" - нетерминал "идентификатор",

"<BUK>" - нетерминал "буква",

"<CIF>" - нетерминал "цифра",

"<TEL>" - нетерминал "тело программы",

"<ODC>" - нетерминал "оператор declare",

"<IPE>" - нетерминал "имя переменной",

"<RZR>" - нетерминал "разрядность",

"<LIT>" - нетерминал "литерал",

"<MAN>" - нетерминал "мантисса",

"<OPA>" - нетерминал "оператор присваивания арифметический",

"<AVI>" - нетерминал "арифметическое выражение",

"<ZNK>" - нетерминал "знак",

"<OEN>" - нетерминал "оператор эпилога программы".

Рис. 6

Для реализации условного варианта задания необходимо в синтаксисе ЯВУ разрешить использование в арифметических выражениях дополнительных операций **умножения** «*****» и **деления** «/». Это разрешение м.б. выполнено небольшим изменением правой части синтаксического правила № 15. При этом, новый вариант синтаксиса м.б. описан так, как на рис 7, на котором дополнительные правки выделены жирным шрифтом.

```
 1.  <PRO> ::= <OPR><TEL><OEN>

 2.  <OPR> ::= <IPR>:PROC_OPTIONS(MAIN);

 3.  <IPR> ::= <IDE>

 4.  <IDE> ::= <BUK> | <IDE><BUK> | <IDE><CIF>

 5.  <BUK> ::= A | B | C | D | E | M | P | X

 6.  <CIF> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

 7.  <TEL> ::= <ODC> | <TEL><ODC> | <TEL><OPA>

 8.  <ODC> ::= DCL_<IPE>_BIN_FIXED(<RZR>); |

               DCL_<IPE>_BIN_FIXED(<RZR>)INIT(<LIT>);

 9.  <IPE> ::= <IDE>

10.  <RZR> ::= <CIF> | <RZR><CIF>

11.  <LIT> ::= <MAN>B

12.  <MAN> ::= 1 | <MAN>0 | <MAN>1

13.  <OPA> ::= <IPE>=<AVI>;

14.  <AVI> ::= <LIT> | <IPE> | <AVI><ZNK><LIT> |

                                        <AVI><ZNK><IPE>

15.  <ZNK> ::= + | - | * | /

16.  <OEN> ::= END_<IPR>
```

Рис. 7

Отметим важную деталь, состоящую в том, что для нашего случая нам не пришлось вводить в синтаксис новые нетерминальные символы, а это, в свою очередь, не потребует изменять состав множества семантических функций (семантических подпрограмм). Достаточно будет изменить алгоритм только в существующих семантических подпрограммах, соответствующих существующим нетерминальным символам.

Синтаксические правки

Вносим коррекции, учитывающие обработку операций умножения и деления в существующие семантические подпрограммы **ZNK** и/или **AVI**. Состав семантических подпрограмм не меняется, ввиду отсутствия необходимости.

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту д)

Методика предварительной подготовки и ввода коррекций в исходный текст программы-макета **Komppl.c** описана в **Пособие.**

Если появились проблемы и/или вопросы, то идем с ними за консультацией к преподавателю.

Действия по пункту е)

Отладка программы макета компилятора ЯВУ, модифицированного правками, производится студентами с опорой на собственный опыт написания и отладки Linux-приложений.
Отладку можно считать законченной в случае, если в процессе экспериментальной проверки работоспособности после запуска отлаживаемой программы компилятора ЯВУ на выходе генерируется ассемлеровский эквивалент входного персонального задания, который совпадает с эталонным вариантом, согласованным с преподавателем на этапе постановки задачи (в ТЗ).
Работа отлаженной версии компилятора предъявляется (демонстрируется) преподавателю с одновременным вручением отчета для проверки/принятия.

В крайнем случае, если появились проблемы и/или вопросы, то идем с ними за как в консультацией к преподавателю.

Действия по пункту ж)

Отчет по результатам составляется с учетом «**Требований к содержанию отчета**», приведенных, как в разделе 4 **Пособия**, так и ниже:

« . . .
Отчет должен соответствовать рекомендациям по оформлению студенческих курсовых работ, размещенных на сайте университета, и в общем случае должен содержать следующие разделы:

- *описания примера из личного задания (указать формулировку личного задания),*
- *постановки задачи модификации программы-макета (описание выбранного способа модификации программы-макета),*

- *содержащие перечень и пояснения модификаций состава и (или) структуры БД компилятора,*
- *перечень и пояснения модификаций алгоритма программы-макета компилятора (возможные формы описаний в виде: блок-схемы, текста на Си с комментариями, а в очевидных случаях обычным текстом),*
- *выводы по результатам работы (указать позитивные и негативные моменты, проявившиеся в процессе работы, с общей личной оценкой проделанной работы).*

. . . »

Литература

1. Расторгуев В.Я. Разработка элементов учебной системы программирования. Компилятор с языка высокого уровня [электронный ресурс]: учебное пособие, СПбПУ, Спб., 2012